Redes generativas adversarias (GAN): fundamentos, práctica en Google Colab y análisis de resultados

Juan Nicolás Romero Estupiñan

Universidad de Cundinamarca

Inteligencia Artificial - Conceptos

Docente: Paul Alexander Diaz Montaña

Noviembre de 2026

Redes generativas adversarias (GAN): fundamentos, práctica en Google Colab y análisis de resultados

Las redes generativas adversarias (GAN, por sus siglas en inglés) son una familia de modelos de aprendizaje profundo capaces de aprender la distribución de un conjunto de datos y producir muestras nuevas que se parecen a los datos originales. Desde su propuesta (Goodfellow et al., 2014), se han usado para generar imágenes y datos sintéticos, hasta el punto de producir rostros de celebridades que no existen (Knight, 2017; Nemire, 2017).

Este informe presenta los conceptos fundamentales de las GAN, describe una práctica sencilla desarrollada en Google Colab con el conjunto de dígitos manuscritos MNIST, reporta los resultados obtenidos y los analiza a la luz de la teoría. El trabajo se apoya en las dos lecturas recomendadas ("Artificial Intelligence + GANs", s. f.; Dietz, 2017) y en fuentes primarias publicadas en línea. Se organiza en cuatro partes: conceptos clave, descripción de la práctica, resultados con su análisis, y conclusiones.

Conceptos clave

¿Qué es una red generativa adversaria?

Una GAN está formada por dos redes separadas, el generador y el discriminador, y plantea el problema del aprendizaje no supervisado como un juego entre ambas (Dietz, 2017). Goodfellow et al. (2014) la propusieron como un procedimiento en el que se entrenan a la vez un modelo generativo, que captura la distribución de los datos, y un modelo discriminativo, que estima si una muestra proviene de los datos de entrenamiento o del generador. El generador se entrena para maximizar la probabilidad de que el discriminador se equivoque, lo que se describe como un juego minimax de dos jugadores.

Dietz (2017) explica por qué esta idea es importante: el aprendizaje supervisado requiere una etiqueta por cada muestra y anotar datos con precisión suele ser poco práctico, mientras que el aprendizaje no supervisado permite entrenar con datos sin anotar y luego afinar el modelo con un conjunto etiquetado mucho más pequeño. Según este autor, las GAN son una de las líneas más prometedoras de investigación en aprendizaje no supervisado.

Estructura: generador y discriminador

Siguiendo a Dietz (2017), una GAN se descompone en tres elementos (Tabla 1). Los datos son muestras de una distribución real y pueden ser imágenes, voz o lecturas de sensores. El generador recibe un código, normalmente ruido aleatorio, y lo transforma en una muestra de datos; su meta es producir muestras variadas que pertenezcan a la distribución real. El discriminador recibe una muestra y la clasifica como real o generada; su meta es distinguirlas con alta precisión. La Figura 1 resume cómo se conectan estos componentes.

Tabla 1

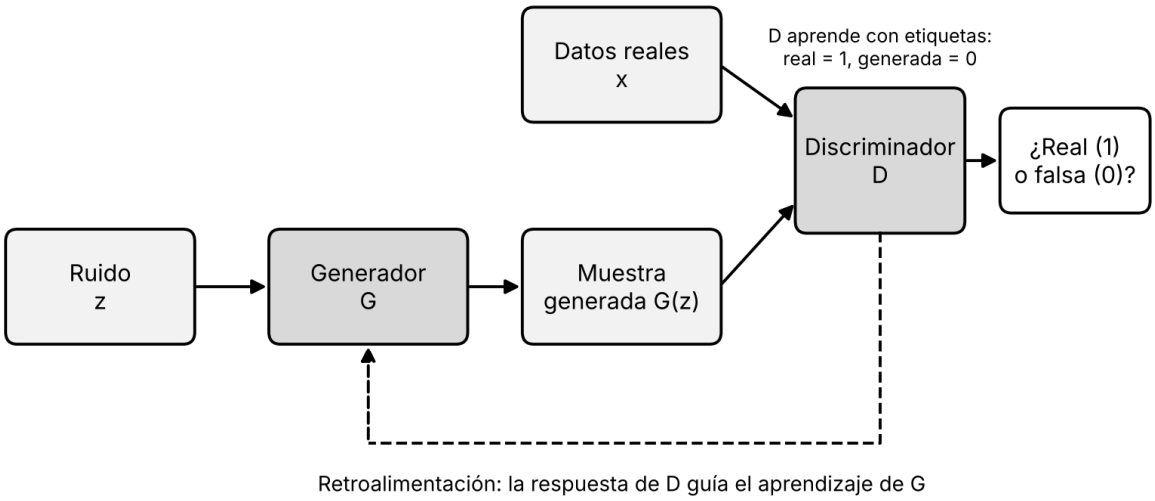
Componentes de una red generativa adversaria

Componente	**Entrada**	**Salida**	**Objetivo**
Datos reales	—	Muestras de la distribución real	Servir de referencia de lo que es real
Generador (G)	Vector de ruido aleatorio z	Muestra sintética	Producir muestras variadas que parezcan reales
Discriminador (D)	Una muestra, real o generada	Probabilidad de que sea real	Distinguir con precisión lo real de lo generado

Nota. Elaboración propia a partir de Dietz (2017) y Goodfellow et al. (2014).

Figura 1

Estructura general de una GAN



La línea punteada indica que el generador solo aprende a partir de la respuesta del discriminador.

Un detalle importante es que solo el discriminador tiene acceso directo a los datos reales; el generador aprende únicamente a partir de la información que le devuelve el discriminador (Dietz, 2017).

Aprendizaje adversarial

El aprendizaje adversarial se expresa con la función de valor siguiente (Goodfellow et al., 2014), donde D(x) es la probabilidad que el discriminador asigna a que x sea real y G(z) es la muestra producida a partir del ruido z. El discriminador intenta maximizar V y el generador intenta minimizarla.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x \sim p_{\text{datos}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

Goodfellow et al. (2014) mostraron que, en teoría, este juego tiene una solución única en la que el generador reproduce la distribución de los datos de entrenamiento y el discriminador devuelve 1/2 para cualquier entrada; es decir, solo puede adivinar con 50 % de probabilidad (Dietz, 2017). Cuando ambas redes son perceptrones multicapa, todo el sistema puede entrenarse con retropropagación (Goodfellow et al., 2014).

En la práctica, el entrenamiento alterna dos pasos (Dietz, 2017). En el primero se entrena el discriminador con un lote de imágenes reales etiquetadas como reales y un lote de imágenes generadas etiquetadas como falsas. En el segundo se entrena el generador para que el discriminador clasifique sus imágenes como reales; aquí el discriminador se mantiene congelado, pero los gradientes de su función de pérdida se propagan hacia atrás a través de la red combinada hasta el generador, que actualiza sus pesos. Además, suele reemplazarse el objetivo del generador por maximizar log D(G(z)) en lugar de minimizar log(1 − D(G(z))), porque ofrece mejores gradientes al inicio del entrenamiento (Goodfellow et al., 2014); esta es la variante usada en la práctica.

Dietz (2017) propone la analogía de un combate de boxeo: ambas redes son boxeadores, el discriminador aprende de un entrenador (los datos reales) y el generador solo puede aprender de su compañero de sparring, es decir, del propio discriminador. Tras muchas rondas ambos quedan bien preparados y, en el caso ideal, un combate entre ellos tendría probabilidades de 50/50. El mismo autor observa que en la vida real el aprendizaje adversarial parece colaborativo (al aprender una canción en la guitarra, el oído hace de discriminador y las manos de generador), mientras que en el software se implementa como una competencia.

Dificultades del entrenamiento

Lograr que generador y discriminador converjan a un equilibrio es una de las principales dificultades de las GAN, y la mayoría de las veces no ocurre de forma limpia (Dietz, 2017). Este autor describe tres fallos frecuentes. Primero, el discriminador se vuelve demasiado fuerte demasiado rápido y clasifica las muestras generadas como falsas con tanta seguridad que sus gradientes no ofrecen nada que el generador pueda aprender. Segundo, el generador aprende debilidades específicas del discriminador y las explota para engañarlo, en lugar de representar la verdadera distribución de los datos. Tercero, el generador aprende solo un subconjunto muy pequeño de la distribución; el caso extremo es que produzca la misma muestra para cualquier entrada, fenómeno conocido como colapso de modos. Estos tres fallos guiaron la elección de las métricas de evaluación de la práctica.

Variantes y aplicaciones

Radford et al. (2015) introdujeron las GAN convolucionales profundas (DCGAN), una familia de redes convolucionales con restricciones arquitectónicas específicas cuyo generador y discriminador aprenden jerarquías de características, desde partes de objetos hasta escenas completas; además, esas características pudieron reutilizarse en otras tareas. Karras et al. (2017) propusieron el crecimiento progresivo: ambas redes empiezan con baja resolución y se agregan capas con detalles más finos durante el entrenamiento, lo que lo hace más rápido y estable y permitió generar rostros de 1024 × 1024 píxeles a partir de CelebA.

Este trabajo está detrás del caso de los rostros de celebridades que no existen. Según Nemire (2017), el equipo de NVIDIA creó el conjunto CelebA-HQ, con 30 000 imágenes de 1024 × 1024 píxeles, y entrenó la red durante 20 días en una sola GPU Tesla P100; la publicación no reporta métricas cuantitativas y presenta como resultado principal la calidad

visual de las imágenes. Knight (2017) añade que los rostros resultaron convincentes en general, aunque algunos tenían fallos como una ceja ausente o dientes mal ubicados, y usa esos errores para argumentar que el sistema produce imágenes llamativas sin comprender el mundo real.

Más allá de las imágenes, Dietz (2017) señala que, tras entrenar una GAN, el discriminador suele usarse como modelo base para transferencia de aprendizaje y ajuste fino, y el generador como fuente de datos para entrenar otro modelo. Finalmente, la facilidad para producir rostros realistas invita a reflexionar sobre su posible uso indebido, por ejemplo, en perfiles falsos, aspecto que debe considerarse al aplicar estas técnicas.

Descripción de la práctica

Para el desarrollo de esta práctica se utilizó Claude (Anthropic, 2026) como herramienta de apoyo para generar el código base del notebook y redactar un borrador del informe. El notebook fue ejecutado en Google Colab y los resultados reportados provienen de dicha ejecución.

Objetivo y entorno

La práctica consistió en entrenar una GAN simple que aprenda a generar dígitos manuscritos a partir del conjunto MNIST. Se desarrolló en un notebook de Google Colab con Python y PyTorch (archivo GAN_MNIST_Colab.ipynb, Apéndice A), ejecutado en una GPU Tesla T4 proporcionada por Colab. Se fijó la semilla aleatoria en 42 para facilitar la reproducibilidad.

Datos

Se utilizó MNIST, que contiene 60 000 imágenes de entrenamiento y 10 000 de prueba de dígitos escritos a mano del 0 al 9, de 28 × 28 píxeles en escala de grises (LeCun et al., 2010). Las imágenes se normalizaron al rango [−1, 1] para coincidir con la salida de la función Tanh del

generador. Las etiquetas no se emplearon para entrenar la GAN, de modo que el aprendizaje fue no supervisado; solo se usaron después en un clasificador auxiliar de evaluación.

Arquitectura

Ambas redes son perceptrones multicapa (Tabla 2). El generador transforma un vector de ruido de 100 dimensiones en una imagen de 784 valores (28 × 28). El discriminador recibe la imagen aplanada y devuelve un logit, que se convierte en probabilidad de ser real mediante la función sigmoide. En el discriminador se añadió Dropout para evitar que se vuelva demasiado fuerte demasiado pronto, uno de los fallos descritos por Dietz (2017).

Tabla 2

Arquitectura del generador y del discriminador

Elemento	**Generador**	**Discriminador**
Entrada	Vector z de 100 dimensiones	Imagen de 28 × 28 (784 valores)
Capas ocultas	256 → 512 → 1024 neuronas	1024 → 512 → 256 neuronas
Activación	LeakyReLU (0.2)	LeakyReLU (0.2)
Regularización	Ninguna	Dropout (0.3) tras cada capa oculta
Salida	784 valores con Tanh	1 logit (la sigmoide se aplica en la pérdida)
Parámetros entrenables	1 486 352	1 460 225

Nota. El número de parámetros se calculó a partir de las dimensiones de las capas.

Procedimiento de entrenamiento

El entrenamiento siguió el esquema alternado descrito en el marco conceptual, con un paso del discriminador y uno del generador por cada lote (Tabla 3). Para el discriminador se minimizó la entropía cruzada binaria sobre lotes de imágenes reales (etiqueta 1) y generadas (etiqueta 0); las imágenes generadas se separaron del grafo de cálculo para que este paso no modificara al generador. Para el generador se usó la pérdida no saturante, es decir, se le pidió

producir imágenes que el discriminador clasifique como reales. Ambas redes se optimizaron con Adam (Kingma y Ba, 2014). Para visualizar la evolución se usó siempre el mismo ruido fijo y se guardaron muestras en las épocas 1, 5, 10, 25 y 50.

Tabla 3

Hiperparámetros de entrenamiento

Hiperparámetro	**Valor**
Épocas	50
Tamaño de lote	128
Dimensión del ruido (z)	100
Optimizador	Adam
Tasa de aprendizaje	0.0002
β_1, β_2 de Adam	0.5, 0.999
Función de pérdida	Entropía cruzada binaria (con logits)
Semilla aleatoria	42

Nota. Valores definidos en la primera celda del notebook.

Métricas de evaluación

Se emplearon cinco formas de evaluación, elegidas para cubrir los fallos descritos por Dietz (2017): (a) las pérdidas de ambas redes por época; (b) la probabilidad media que el discriminador asigna a imágenes reales, D(x), y a imágenes generadas, D(G(z)); (c) la exactitud del discriminador sobre las 10 000 imágenes reales de prueba y sobre 10 000 imágenes recién generadas; (d) la distribución de dígitos generados según un clasificador auxiliar (red convolucional entrenada 2 épocas con las etiquetas de MNIST), junto con su confianza media, para detectar colapso de modos; y (e) una interpolación en el espacio latente. La métrica (d) es solo una aproximación y no sustituye a métricas estándar como FID, que no se calcularon.

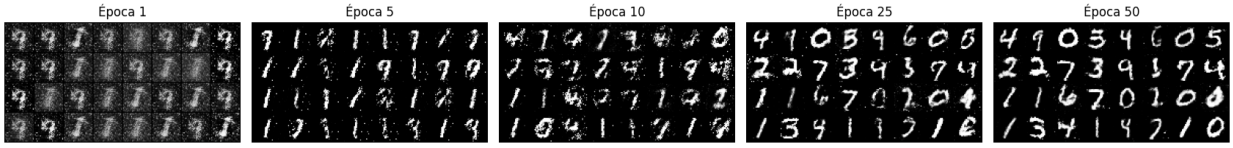
Resultados obtenidos

Esta sección reúne los resultados que produjo el notebook. Las figuras se generan automáticamente y los valores numéricos se resumen en la Tabla 4.

La Figura 2 muestra las mismas 32 muestras, generadas con el mismo ruido fijo, en las épocas 1, 5, 10, 25 y 50. En la época 1 el generador produce manchas grises con ruido y casi la misma forma en todas las muestras. En la época 5 aparecen trazos blancos definidos, pero con predominio de formas parecidas a un 1 o a un 7. En la época 10 hay más variedad, aunque con mucho ruido alrededor de los trazos. Desde la época 25 se reconocen dígitos variados (por ejemplo 4, 0, 5, 6, 2, 7 y 3) y en la época 50 los trazos son más limpios.

Figura 2

Evolución de las muestras generadas durante el entrenamiento



La Figura 3 presenta 64 dígitos generados al finalizar el entrenamiento. La mayoría se reconoce con facilidad, aunque algunos presentan defectos: trazos borrosos o con manchas, formas ambiguas entre dos dígitos y trazos incompletos.

Figura 3

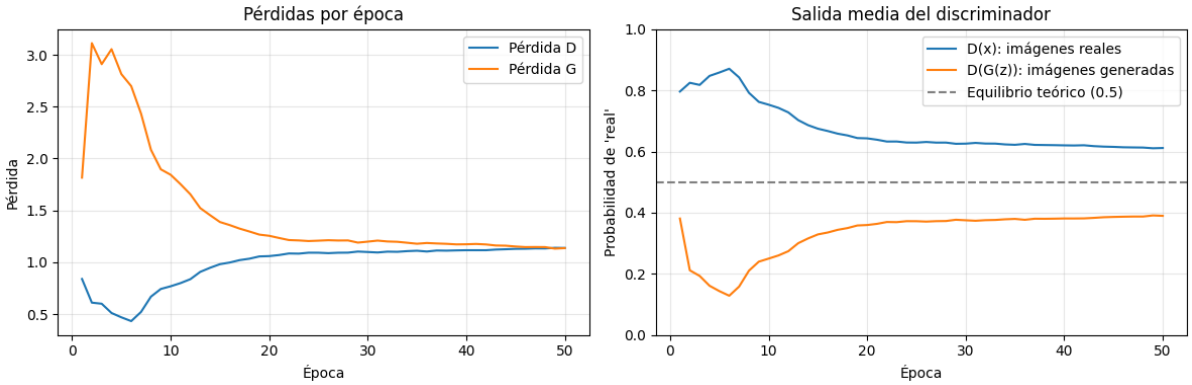
Dígitos generados tras 50 épocas



La Figura 4 muestra las pérdidas por época y la salida media del discriminador. Se distinguen dos fases. Hasta la época 6, la pérdida de D baja hasta 0.433 mientras la de G sube hasta 3.112 en la época 2 y se mantiene alta; en ese tramo D(x) llega a 0.871 y D(G(z)) cae a 0.128. Después ambas curvas se acercan, se estabilizan desde alrededor de la época 25 y en la época 50 las dos pérdidas valen casi lo mismo (1.138 y 1.137). D(x) y D(G(z)) se aproximan a 0.5 sin alcanzarlo: 0.612 y 0.390 en la época 50.

Figura 4

Pérdidas y salida media del discriminador por época



Nota. La línea punteada marca el equilibrio teórico de 0.5.

La Tabla 4 resume las métricas finales de la práctica.

Tabla 4

Métricas finales de la práctica

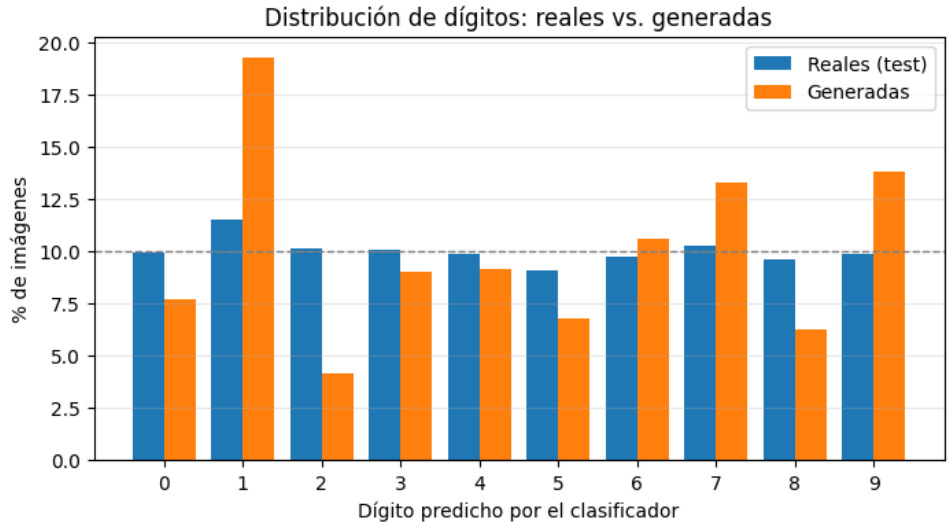
Métrica	**Valor**
Dispositivo de ejecución	GPU Tesla T4 (Google Colab)
Tiempo de entrenamiento (min)	13.2
Pérdida de D, época 50	1.138
Pérdida de G, época 50	1.137
D(x) medio, época 50	0.612
D(G(z)) medio, época 50	0.390
Exactitud de D en imágenes reales de prueba (%)	50.7
Exactitud de D en imágenes generadas (%)	92.9
Exactitud del clasificador auxiliar en MNIST de prueba (%)	98.66
Confianza media del clasificador, imágenes reales	0.983
Confianza media del clasificador, imágenes generadas	0.920
Imágenes generadas con confianza menor a 0.9 (%)	23.0
Dígito generado más frecuente (% y dígito)	19.3 % (dígito 1)
Dígito generado menos frecuente (% y dígito)	4.2 % (dígito 2)

Nota. Valores tomados del resumen final del notebook ejecutado. Las exactitudes de D y del clasificador se calcularon sobre las 10 000 imágenes de prueba y 10 000 imágenes generadas.

La Figura 5 compara la distribución de dígitos entre las imágenes reales de prueba y las generadas, según el clasificador auxiliar. Los 10 dígitos aparecen en las imágenes generadas, pero con frecuencias desiguales: el 1 es el más frecuente (19.3 %) y el 2 el menos frecuente (4.2 %), mientras que en las imágenes reales todos los dígitos están cerca del 10 %. Los dígitos 7 (13.3 %) y 9 (13.8 %) también superan el 10 %, y los dígitos 0, 2, 5 y 8 quedan por debajo del 8 %.

Figura 5

Distribución de dígitos en imágenes reales y generadas

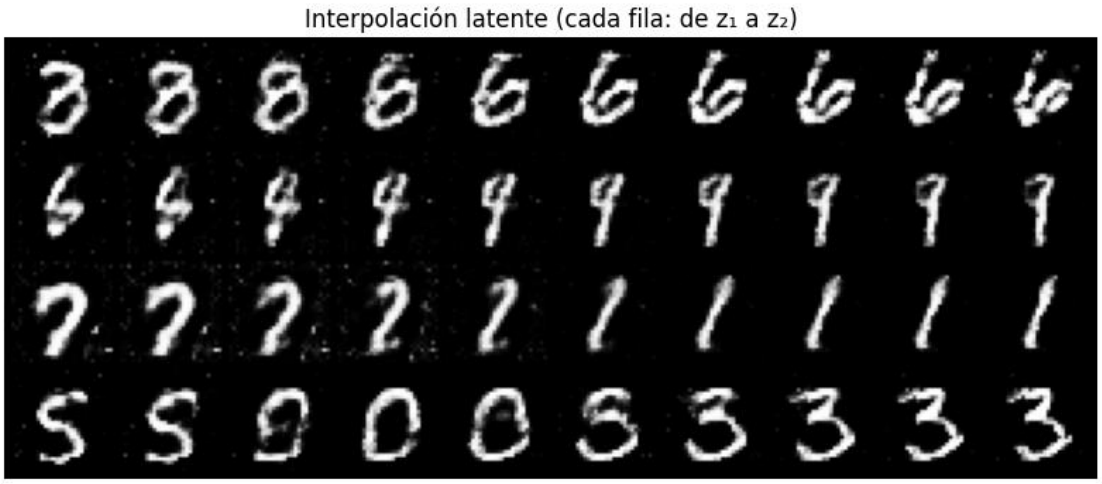


Nota. La línea punteada marca el 10 %.

La Figura 6 muestra cuatro interpolaciones entre pares de vectores de ruido. En las filas 1 a 3 las transiciones son graduales: un 3 pasa por un 8 hasta llegar a un 6, una forma parecida a un 6 se convierte en un 9, y un 7 pasa por un 2 hasta terminar en un 1. La fila 4 es menos regular: va de un 5 a un 3 pasando por formas ambiguas.

Figura 6

Interpolación en el espacio latente



Nota. Cada fila va de un vector z_1 a un vector z_2.

Análisis de resultados

Calidad y evolución de las muestras

Que el generador mejore con las épocas es lo que la teoría anticipa: G parte de pesos aleatorios y solo aprende de los gradientes que le devuelve D, de modo que sus muestras deberían volverse más reconocibles a medida que D aprende a señalar qué las diferencia de las reales (Dietz, 2017). La secuencia de la Figura 2 confirma esa expectativa: la mejora fue progresiva y el salto más claro hacia dígitos reconocibles ocurrió entre las épocas 10 y 25. La Figura 3 permite además juzgar defectos que el generador no corrige, como trazos incompletos o dígitos ambiguos; son comparables, a menor escala, con los rostros con una ceja ausente o dientes mal ubicados que reportó Knight (2017). En este caso, los defectos fueron trazos borrosos, manchas y formas ambiguas en una parte de las 64 muestras.

Equilibrio entre generador y discriminador

En el equilibrio teórico, D(x) y D(G(z)) tienden a 0.5 (Dietz, 2017; Goodfellow et al., 2014), pero alcanzarlo exactamente es poco común. Una brecha amplia entre ambos valores, con D(x) cercano a 1 y D(G(z)) cercano a 0, indicaría que el discriminador domina y que el generador recibe pocas señales útiles, que es el primer fallo descrito por Dietz (2017); valores próximos entre sí indicarían un combate parejo. En este experimento, D(x) fue 0.612 y D(G(z)) fue 0.390 al final del entrenamiento (Tabla 4), y la exactitud de D sobre imágenes de prueba reales y generadas fue 50.7 % y 92.9 %, respectivamente. Ambas probabilidades se acercaron a 0.5 respecto de las primeras épocas: la brecha entre ellas bajó de 0.743 en la época 6 a 0.222 en la época 50. Aun así, el discriminador conservó una ventaja moderada, pues en evaluación identificó como falsas el 92.9 % de las imágenes generadas. Esa cifra no indica un discriminador muy fuerte en general, ya que solo clasificó como reales el 50.7 % de las imágenes reales con el umbral de 0.5; D tiende a responder «falsa» y su exactitud global, el promedio de ambas, fue 71.8 %, bastante por encima del 50 % ideal. Por tanto, el generador mejoró, pero no logró engañar al discriminador de forma sistemática. No se observó el primer fallo descrito por Dietz (2017) en su forma extrema: G siguió mejorando hasta la época 50 y su pérdida se estabilizó en lugar de crecer. Como D usa Dropout durante el entrenamiento y se evalúa sin él, las cifras de entrenamiento y de evaluación no coinciden exactamente (D(x) de 0.553 y D(G(z)) de 0.331 en evaluación).

Interpretación de las curvas de pérdida

A diferencia de un clasificador convencional, en una GAN la pérdida no tiene por qué descender de forma monótona, porque cada red mejora frente a un oponente que también cambia. Por eso la pérdida por sí sola no mide la calidad de las imágenes y debe leerse junto con las

muestras y con D(x) y D(G(z)). Una pérdida de G que crece de forma sostenida mientras la de D tiende a cero sería consistente con un discriminador demasiado fuerte. En este experimento, la pérdida de G subió al inicio (hasta 3.112) y luego bajó y se estabilizó cerca de 1.14, mientras que la de D pasó de 0.433 en la época 6 a 1.138; las dos curvas convergieron y no hubo un crecimiento sostenido de la pérdida de G con la de D cercana a cero. Con D(x) y D(G(z)) iguales a 0.5, la pérdida de D valdría 2 ln 2, es decir, unos 1.386; el valor final de 1.138 queda por debajo, lo que concuerda con un discriminador que aún conserva algo de ventaja.

Diversidad y colapso de modos

El tercer fallo descrito por Dietz (2017) es que el generador aprenda solo un subconjunto de la distribución. Si la distribución de dígitos generados se parece a la de las imágenes reales, con cada dígito cerca del 10 %, hay evidencia de diversidad; si pocos dígitos concentran la mayoría de las muestras, hay colapso parcial o total de modos. Aquí el dígito más frecuente (el 1) representó 19.3 % y el menos frecuente (el 2) 4.2 % de las imágenes generadas (Tabla 4 y Figura 5). La confianza media del clasificador en las imágenes generadas (0.920) frente a las reales (0.983), junto con el 23.0 % de imágenes generadas con confianza menor a 0.9, indica que los dígitos generados son en general reconocibles, pero menos nítidos que los reales; hay que tener presente que un clasificador puede estar muy seguro ante una imagen mal formada. No hubo colapso total de modos, porque los 10 dígitos aparecen y el menos frecuente alcanza 4.2 %. Sí hay un desbalance parcial: el 1 casi duplica su frecuencia real y el 2 queda a menos de la mitad. Parte de ese desbalance podría deberse al clasificador, que pudo asignar formas ambiguas o de trazo fino a ciertas clases, por lo que la cifra es solo aproximada.

Espacio latente

Las transiciones graduales entre dos dígitos al recorrer la recta que une dos vectores z sugieren que el generador aprendió una representación continua y no memorizó ejemplos; saltos bruscos o imágenes sin forma en puntos intermedios indicarían lo contrario. En la Figura 6, la mayoría de las filas muestra transiciones graduales sin saltos bruscos ni ruido en los puntos intermedios, lo que apoya la idea de un espacio latente continuo; la fila 4 pasa por formas ambiguas, lo que indica que algunas regiones del espacio latente producen dígitos poco definidos.

Limitaciones y mejoras

Los resultados deben interpretarse con cautela. Se usaron redes densas, más simples que las arquitecturas convolucionales de las DCGAN (Radford et al., 2015); el entrenamiento se limitó a 50 épocas y a una sola semilla, por lo que no se midió la variabilidad entre ejecuciones; y la evaluación se apoyó en un clasificador auxiliar y en inspección visual, sin métricas estándar como FID. Como mejoras posibles se proponen una arquitectura DCGAN, repetir el entrenamiento con varias semillas y calcular una métrica cuantitativa de calidad y diversidad.

Conclusiones

La práctica con MNIST permitió observar de forma directa el entrenamiento adversarial: tras 50 épocas (13.2 minutos en una GPU Tesla T4), el generador pasó de manchas sin forma a dígitos en su mayoría reconocibles, pero el discriminador conservó ventaja, pues identificó como falsas el 92.9 % de las imágenes generadas, y la distribución de dígitos fue desigual (de 4.2 % para el 2 a 19.3 % para el 1), lo que indica una diversidad solo parcial.

El análisis mostró que evaluar una GAN exige mirar varias señales a la vez (muestras, salidas del discriminador y diversidad), porque la pérdida por sí sola no indica calidad y porque

fallos como el colapso de modos solo se detectan comparando la distribución de lo generado con la real.

La comparación con el caso de los rostros de celebridades (Karras et al., 2017; Nemire, 2017) muestra que la misma idea escala a imágenes de alta resolución, pero con arquitecturas y entrenamientos mucho más exigentes, como los 20 días en una GPU reportados por NVIDIA. También deja una lección práctica: las imágenes pueden ser convincentes y aun así contener errores (Knight, 2017). Como trabajo futuro se recomienda probar una DCGAN, repetir los experimentos con varias semillas, usar métricas estándar y considerar el uso responsable de estas técnicas.

Repositorio de github con la práctica

https://github.com/NicolasRomero22/RedesGanPractica

Referencias

Artificial Intelligence + GANs can create fake celebrity faces. (s. f.). *Data Driven Investor*. https://medium.datadriveninvestor.com/artificial-intelligence-gans-can-create-fake-celebrity-faces-44fe80d419f7

Dietz, M. (2017, 6 de febrero). Introduction to GANs: A boxing match b/w neural nets. *Medium*. https://medium.com/@waya.ai/introduction-to-gans-a-boxing-match-b-w-neural-nets-b4e5319cc935

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. y Bengio, Y. (2014). *Generative adversarial networks* (arXiv:1406.2661). arXiv. https://arxiv.org/abs/1406.2661

Karras, T., Aila, T., Laine, S. y Lehtinen, J. (2017). *Progressive growing of GANs for improved quality, stability, and variation* (arXiv:1710.10196). arXiv. https://arxiv.org/abs/1710.10196

Kingma, D. P. y Ba, J. (2014). *Adam: A method for stochastic optimization* (arXiv:1412.6980). arXiv. https://arxiv.org/abs/1412.6980

Knight, W. (2017, 31 de octubre). Meet the fake celebrities dreamed up by AI. *MIT Technology Review*. https://www.technologyreview.com/2017/10/31/148034/meet-the-fake-celebrities-dreamed-up-by-ai/

LeCun, Y., Cortes, C. y Burges, C. J. C. (2010). *MNIST handwritten digit database* [Conjunto de datos]. AT&T Labs. http://yann.lecun.com/exdb/mnist/

Nemire, B. (2017, 30 de octubre). Generating photorealistic images of fake celebrities with artificial intelligence. *NVIDIA Developer Blog*.

https://developer.nvidia.com/blog/generating-photorealistic-fake-celebrities-with-artificial-intelligence

Radford, A., Metz, L. y Chintala, S. (2015). *Unsupervised representation learning with deep convolutional generative adversarial networks* (arXiv:1511.06434). arXiv. https://arxiv.org/abs/1511.06434

Anthropic. (2026). *Claude* (versión Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai